Agenda de Contactos con Acceso Aleatorio

María Belén Mendoza Garcia

1517225

Facultad de Ingeniería, Universidad Rafael Landívar

Manejo e Implementación de Archivos, Sección 01

Profesor Manuel Rojas

9 de noviembre de 2026

Agenda de Contactos con Acceso Aleatorio

1. Introducción

El presente proyecto consiste en el desarrollo de una aplicación de escritorio llamada Agenda de Contactos con Acceso Aleatorio, elaborada en Python como parte del curso de Manejo e Implementación de Archivos.

Su propósito es aplicar los conocimientos relacionados con el almacenamiento y la administración de información mediante archivos, utilizando una estructura que permita acceder a los registros de manera directa, sin necesidad de recorrer todo el contenido del archivo para localizar un contacto.

Para lograrlo, se implementaron registros de tamaño fijo, un archivo de datos en formato binario y un índice que relaciona los identificadores de los contactos con la posición en la que se encuentran almacenados. También se incorporaron funciones para agregar, buscar y modificar información, así como herramientas para exportar los datos, generar copias de seguridad y verificar la integridad de los archivos.

El desarrollo de esta aplicación permitió llevar a la práctica conceptos fundamentales sobre el manejo de archivos, comprendiendo cómo influye la organización de los registros en la forma de almacenar y recuperar información. De esta manera, el proyecto integra diferentes herramientas de programación en una aplicación funcional que facilita la administración de contactos.

2. Objetivos

Objetivo general

Desarrollar una aplicación de escritorio en Python para administrar contactos mediante archivos de acceso aleatorio, utilizando registros de tamaño fijo, un índice para localizar

información y mecanismos que permitan verificar la integridad y conservar los datos almacenados.

Objetivos específicos

- Implementar una estructura de registros de tamaño fijo para almacenar la información de los contactos de manera organizada.
- Aplicar las funciones *seek()* y *tell()* para acceder directamente a las posiciones correspondientes dentro del archivo.
- Desarrollar un índice que permita localizar los registros mediante el identificador de cada contacto.
- Incorporar un mecanismo de verificación de integridad utilizando el algoritmo SHA-256.
- Implementar funciones para exportar la información en formatos JSON y PDF.
- Permitir la generación de copias de seguridad mediante archivos ZIP.
- Desarrollar una interfaz gráfica que facilite la interacción con las diferentes funciones de la aplicación.

3. Descripción general de la aplicación

La aplicación fue desarrollada en Python y utiliza Tkinter para construir su interfaz gráfica. Su función principal es permitir el almacenamiento y la administración de información de contactos mediante archivos de acceso aleatorio.

Cada contacto contiene cinco campos: identificador, nombre, teléfono, correo electrónico y ciudad. Estos datos se almacenan en un archivo binario utilizando una estructura de tamaño fijo, lo que permite calcular la posición de cada registro dentro del archivo.

Entre las operaciones principales se encuentran el registro de nuevos contactos, la búsqueda mediante identificador y la modificación de información existente. Además, la

aplicación cuenta con herramientas complementarias para consultar información de los archivos, exportar los contactos, generar respaldos y comprobar que el contenido del archivo de datos no haya sido alterado.

También se incorporaron traducciones al español y al inglés, junto con validaciones y mensajes que permiten informar al usuario cuando ocurre alguna situación, como ingresar un identificador incorrecto, intentar registrar un contacto duplicado o buscar un contacto que no existe.

La interfaz reúne estas funcionalidades para que el usuario pueda administrar sus contactos sin tener que manipular directamente los archivos que utiliza el sistema.

4. Estructura de los registros

Uno de los aspectos principales del proyecto es la organización de la información mediante registros de tamaño fijo.

Cada contacto ocupa exactamente 128 bytes dentro del archivo *contactos.dat*. Para conseguirlo, se asignó una cantidad determinada de bytes a cada campo, independientemente de la longitud del contenido que introduzca el usuario. La estructura utilizada se presenta en la Tabla 1.

Tabla 1

Estructura del registro de contacto

Campo	**Tamaño**
Identificador	8 bytes
Nombre	40 bytes
Teléfono	15 bytes
Correo electrónico	45 bytes
Ciudad	20 bytes
Tamaño total	128 bytes

Nota. La suma de los cinco campos da como resultado un registro de 128 bytes.

Los datos se almacenan utilizando codificación UTF-8. Cuando el contenido de un campo ocupa menos espacio que el reservado, se completa con espacios hasta alcanzar el tamaño establecido. Esto permite mantener la misma longitud para todos los registros.

La utilización de registros de tamaño fijo es importante porque facilita el cálculo de las posiciones dentro del archivo. Si todos los contactos ocupan exactamente 128 bytes, es posible determinar dónde comienza un registro multiplicando su número de posición por el tamaño de cada registro. Por ejemplo, el primer registro comienza en la posición 0, el segundo en la posición 128, el tercero en la posición 256 y así sucesivamente.

Esta organización permite acceder a la información de forma directa y facilita las operaciones de lectura y modificación.

Figura 1

Definición de los tamaños de los campos y del registro en agenda.py

```
# Tamaño de cada campo en bytes.
campos = {
    "id": 8,
    "nombre": 40,
    "telefono": 15,
    "email": 45,
    "ciudad": 20
}
tamaño_registro = sum(campos.values())
```

5. Implementación del acceso aleatorio con seek() y tell()

Para acceder a los contactos almacenados se utilizaron las funciones *seek()* y *tell()* de Python, las cuales permiten controlar la posición del cursor dentro de un archivo.

La función *tell()* permite conocer la posición actual del cursor, expresada en bytes. Por otra parte, *seek()* permite desplazar el cursor hasta una posición determinada del archivo para realizar una operación de lectura o escritura. En el proyecto, estas funciones se utilizan principalmente durante el registro, la búsqueda y la modificación de contactos.

Posición final del archivo al agregar un contacto

Al agregar un contacto, la aplicación utiliza *seek(0, 2)* para colocar el cursor al final del archivo. Después, mediante *tell()*, obtiene la posición en la que comenzará el nuevo registro. Esta posición permite determinar el número de registro que corresponde al contacto que se está almacenando.

Figura 2

Uso de seek(0, 2) y tell() para determinar la posición final del archivo

```
with open(archivo_contactos, "ab") as archivo:
    archivo.seek(0, 2)
    posicion = archivo.tell()
    numero_registro = posicion // tamaño_registro
    archivo.write(registro)
    posicion_final = archivo.tell()
```

Cálculo de la posición de un registro

Para buscar un contacto, primero se obtiene su número de registro mediante el índice. Luego, se calcula su posición multiplicando dicho número por los 128 bytes que ocupa cada registro.

Figura 3

Cálculo de la posición de un registro a partir de su tamaño

```
numero_registro = int(indice[clave])



with open(archivo_contactos, "rb") as archivo:
    archivo.seek(numero_registro * tamaño_registro)
```

Lectura de un registro

Una vez calculada la posición, se utiliza *seek()* para desplazar el cursor directamente hasta esa ubicación y leer la información correspondiente.

Figura 4

Uso de seek() para leer un registro

```
with open(archivo_contactos, "rb") as archivo:
    archivo.seek(numero_registro * tamaño_registro)
    posicion = archivo.tell()
    registro = archivo.read(tamaño_registro)
```

Modificación de un contacto

El mismo principio se aplica al modificar un contacto. La aplicación localiza la posición del registro y escribe la nueva información en ese lugar, sin necesidad de recorrer todos los contactos anteriores.

Figura 5

Acceso a la posición de un registro para modificar un contacto

```
with open(archivo_contactos, "r+b") as archivo:
    archivo.seek(numero_registro * tamaño_registro)
    posicion = archivo.tell()

    registro_actual = archivo.read(tamaño_registro)

    contacto = decodificar_contacto(registro_actual)
    contacto.update(datos)
    contacto["id"] = int(identificador)

    archivo.seek(posicion)
    archivo.write(codificar_contacto(contacto))
```

La principal ventaja de este procedimiento es que permite trabajar directamente con la ubicación de un registro. En lugar de leer cada contacto desde el principio hasta encontrar el que se necesita, la aplicación puede calcular su posición y acceder a ella. De esta manera, el uso de *seek()* y *tell()* permite aprovechar la estructura de tamaño fijo para realizar operaciones de acceso aleatorio.

6. Funcionamiento del índice

Además del archivo que contiene los registros, la aplicación utiliza un índice almacenado en *indice.json*. Este archivo mantiene una relación entre el identificador de cada contacto y el número de registro en el que se encuentra almacenado. De esta manera, cuando el usuario desea buscar un contacto, no es necesario revisar todos los registros del archivo binario.

El índice funciona como una referencia que permite conocer la ubicación de cada contacto dentro del archivo principal. Por ejemplo, una representación ilustrativa de esta relación se muestra en la Figura 6.

Figura 6

Ejemplo ilustrativo del contenido de indice.json

```
{
    "1": 0,
    "9995": 1,
    "61978": 2,
    "23456": 3
}
```

Nota. Los valores son únicamente ilustrativos.

En este ejemplo, cada identificador está relacionado con el número de registro que le corresponde. El proceso de búsqueda funciona de la siguiente manera:

1. El usuario introduce el identificador del contacto que desea consultar.
2. La aplicación busca ese identificador dentro del índice JSON.
3. Si el identificador existe, obtiene el número de registro asociado.
4. Multiplica el número de registro por 128 para calcular la posición dentro del archivo.
5. Utiliza *seek()* para desplazarse directamente hasta esa posición.
6. Lee el registro y obtiene la información del contacto.

Eficiencia frente a la búsqueda secuencial

La utilización de un índice ofrece una ventaja frente a la búsqueda secuencial. En una búsqueda secuencial sería necesario revisar los registros uno por uno hasta encontrar el identificador solicitado. En cambio, con el índice se puede obtener directamente el número de registro correspondiente y calcular su ubicación.

Esto reduce la cantidad de lecturas necesarias sobre el archivo de contactos y hace que la búsqueda sea más eficiente, especialmente cuando aumenta la cantidad de registros almacenados.

El índice también permite separar la información de referencia de los datos principales. Así, el archivo binario conserva los registros, mientras que el JSON facilita su localización.

7. Verificación de integridad mediante SHA-256

La aplicación incorpora un mecanismo de verificación de integridad para detectar si el archivo de contactos ha sufrido modificaciones desde la última vez que se calculó su huella digital.

Para realizar esta comprobación se utiliza SHA-256, un algoritmo que genera una huella digital a partir del contenido de un archivo. El resultado se representa mediante una cadena de 64 caracteres hexadecimales. En el proyecto, la huella se calcula utilizando el archivo *contactos.dat* y se almacena en *contactos.sha256*.

Cuando se realiza una verificación, la aplicación vuelve a calcular la huella del archivo de contactos y compara el resultado con el valor almacenado anteriormente. Si ambas huellas coinciden, significa que el contenido del archivo no ha cambiado desde que se generó la huella guardada. Si los resultados son diferentes, la aplicación identifica que el archivo fue modificado.

Este mecanismo resulta útil porque permite detectar alteraciones que podrían afectar la información almacenada. Sin embargo, es importante mencionar que SHA-256 no impide que un archivo sea modificado; su función en este proyecto es permitir identificar si su contenido ha cambiado.

Prueba de integridad

Para comprobar el funcionamiento de esta herramienta se realizó una prueba en la que se modificó manualmente el archivo de contactos desde fuera de la aplicación. Posteriormente, se

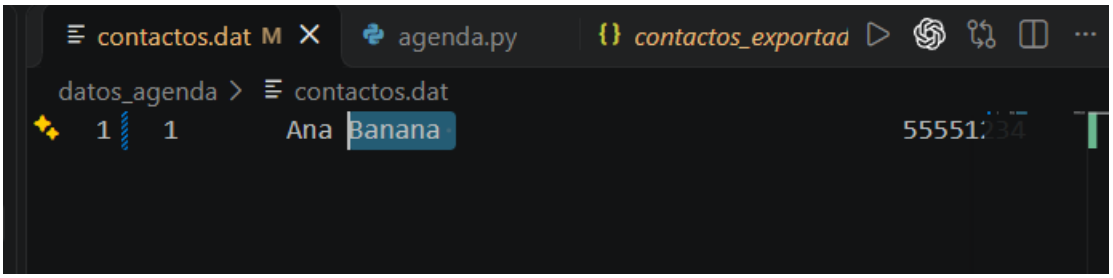
ejecutó la opción de verificación de integridad. Al comparar la huella actual con la que se encontraba almacenada, la aplicación detectó que los resultados eran diferentes y mostró una advertencia indicando que el archivo de contactos pudo haber sido modificado fuera de la aplicación.

Esta prueba permitió comprobar que el mecanismo de verificación cumple con su propósito de detectar cambios en el contenido del archivo.

Figura 7

Archivo de contactos antes o durante la modificación manual

Antes:



Modificado manualmente:

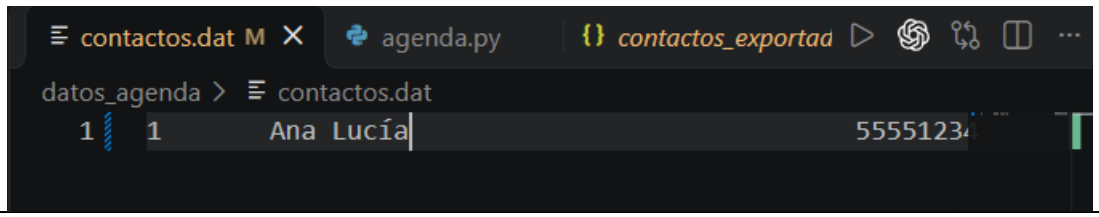
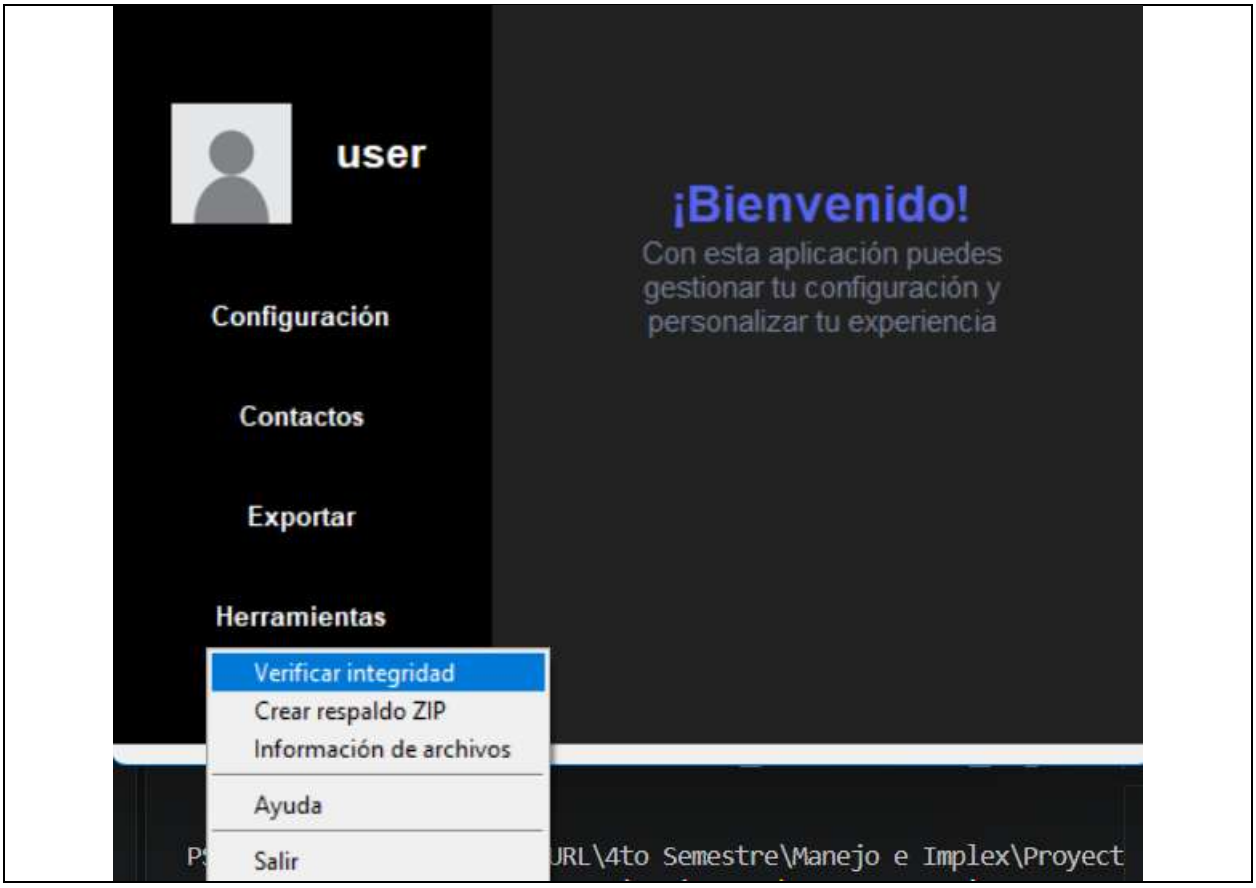
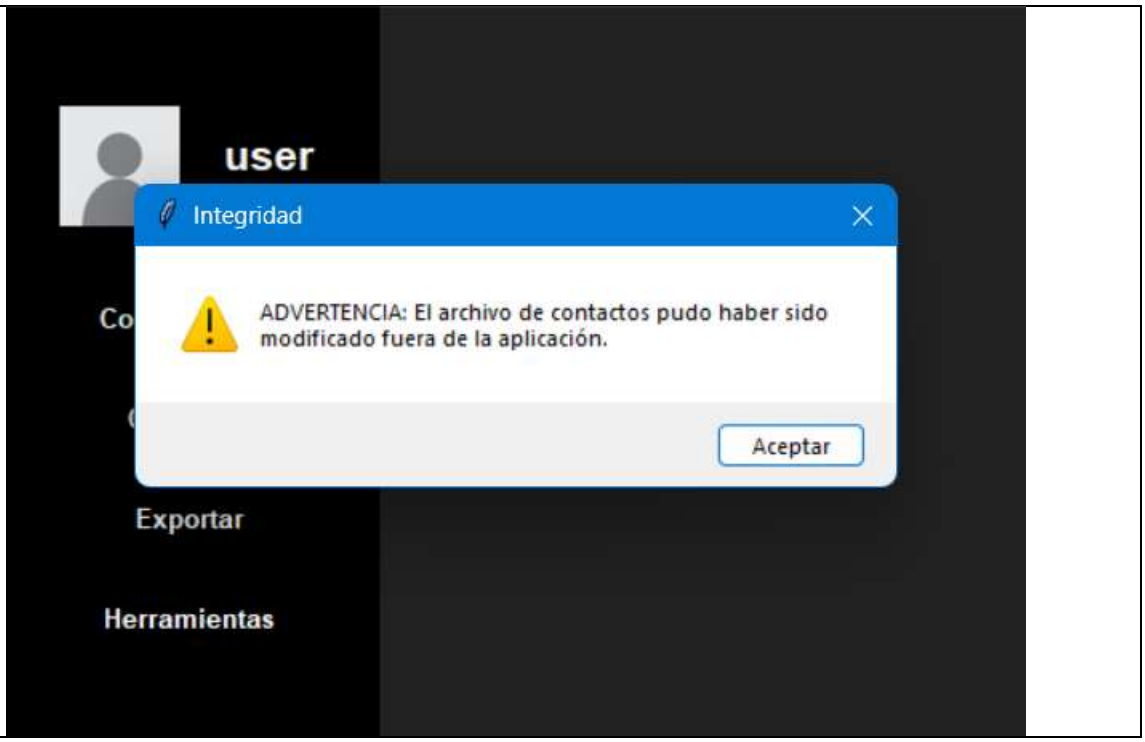


Figura 8

Advertencia de la aplicación al detectar que el archivo fue alterado





8. Exportación de información

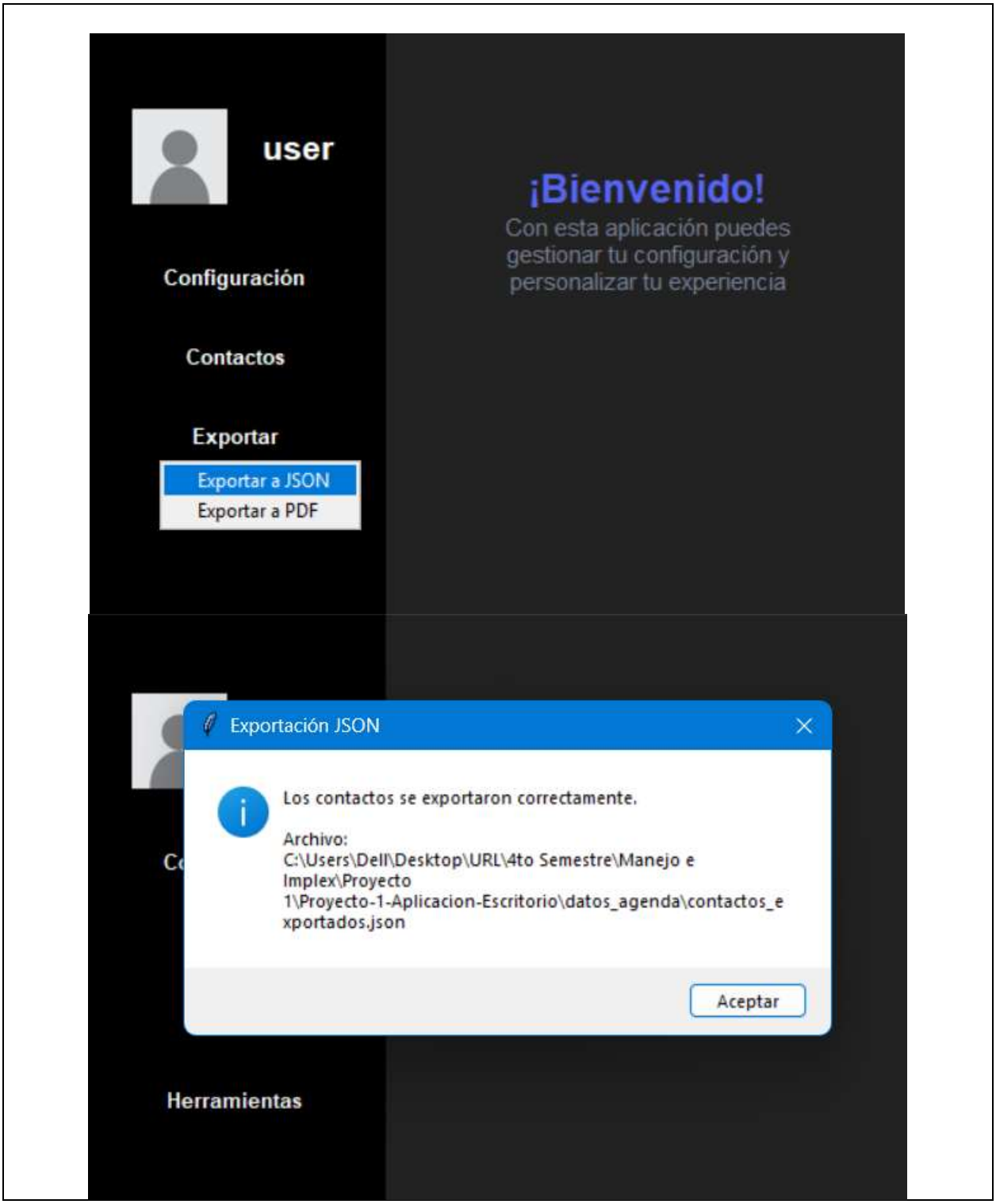
La aplicación incorpora funciones para generar exportaciones de la información almacenada en la agenda. Una de ellas permite generar un archivo JSON, que organiza los datos de los contactos en una estructura que puede ser consultada y utilizada posteriormente por otros programas. Este formato facilita el intercambio de información y permite conservar una representación estructurada de los registros.

También se implementó la generación de un reporte en PDF. Este formato permite presentar la información en un documento que puede visualizarse, almacenarse o compartirse sin necesidad de abrir la aplicación.

Estas opciones amplían las posibilidades de uso de la agenda, ya que permiten conservar la información en formatos distintos al archivo binario utilizado internamente por el sistema.

Figura 9

Exportación de los contactos a JSON



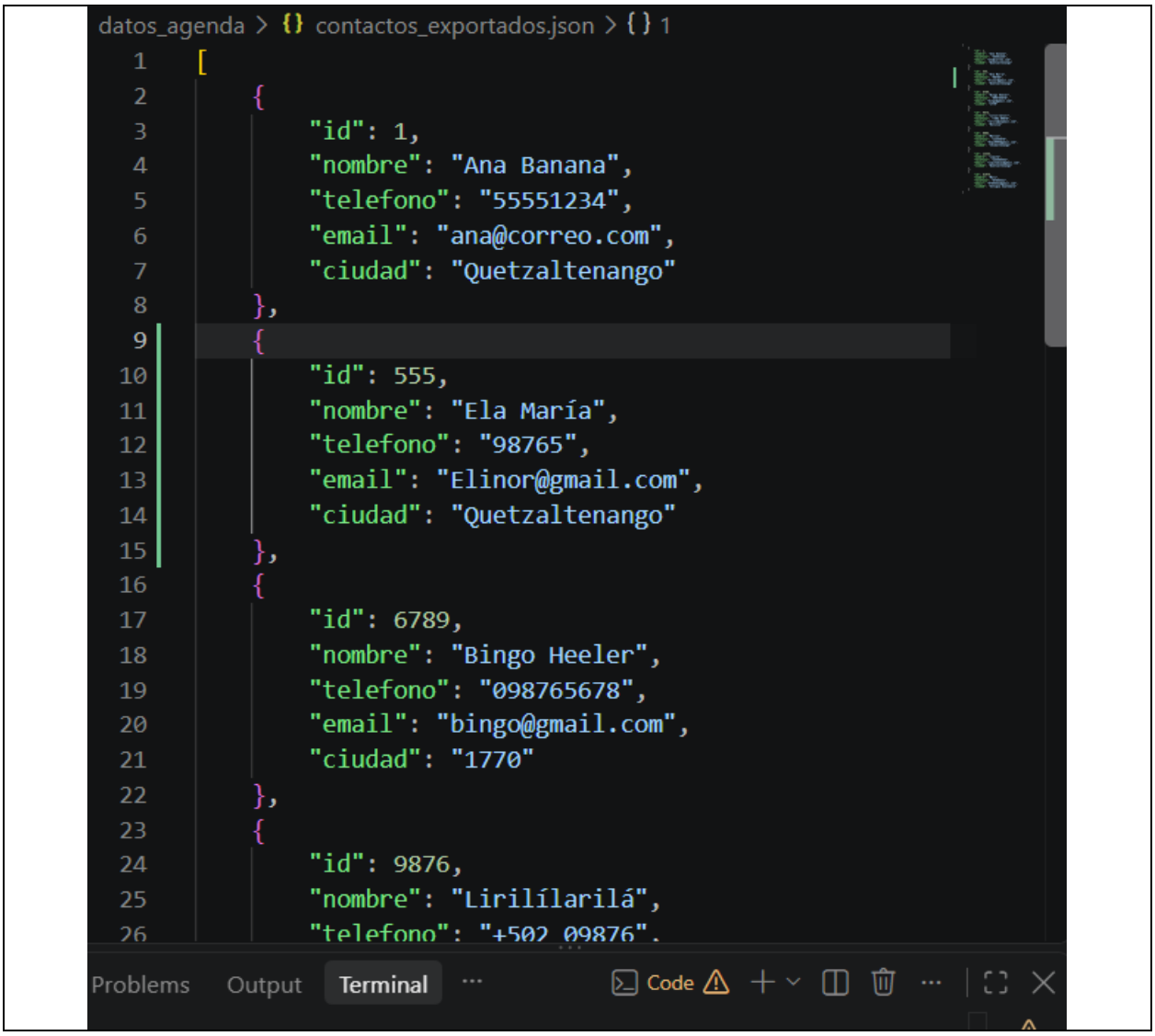
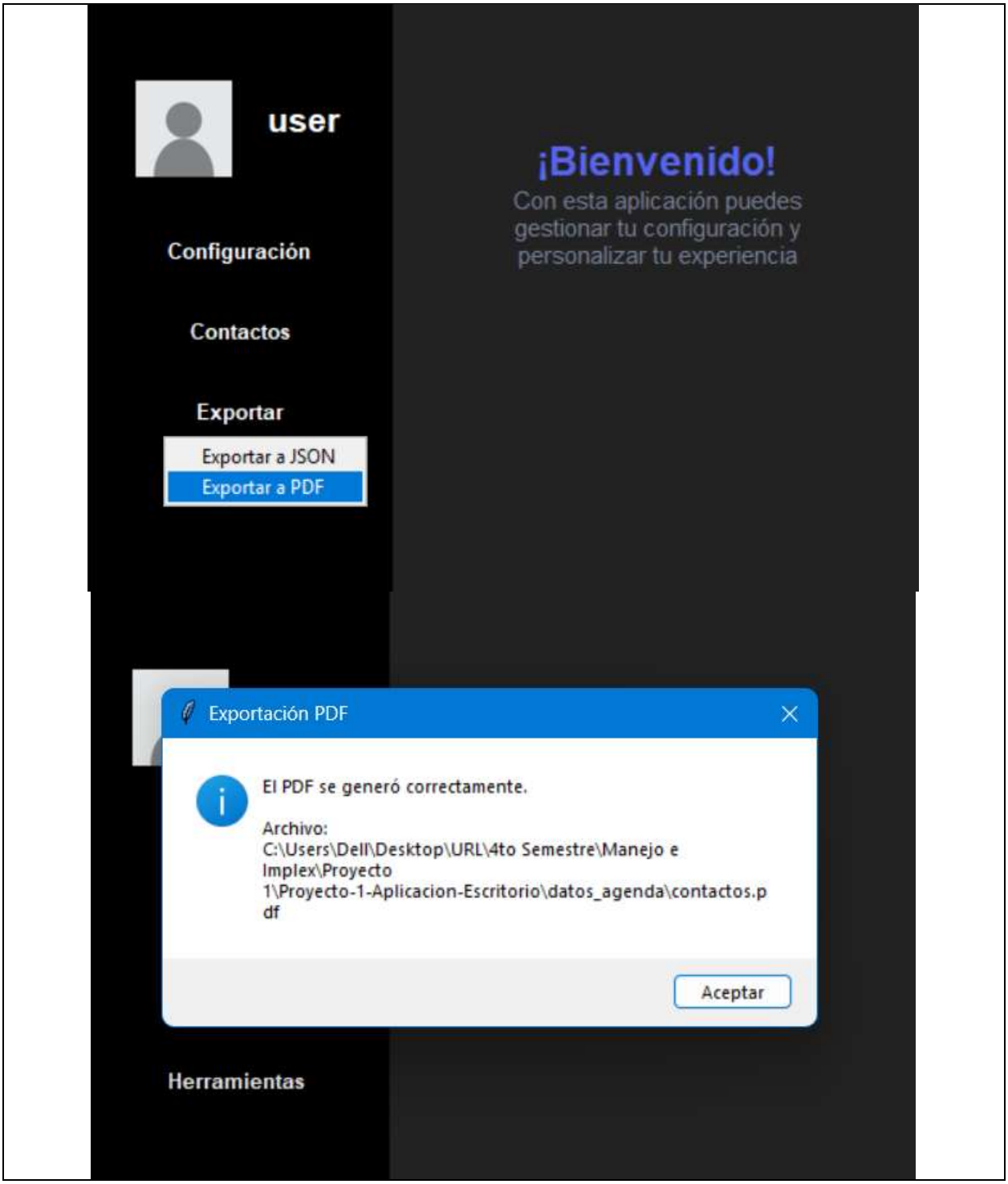
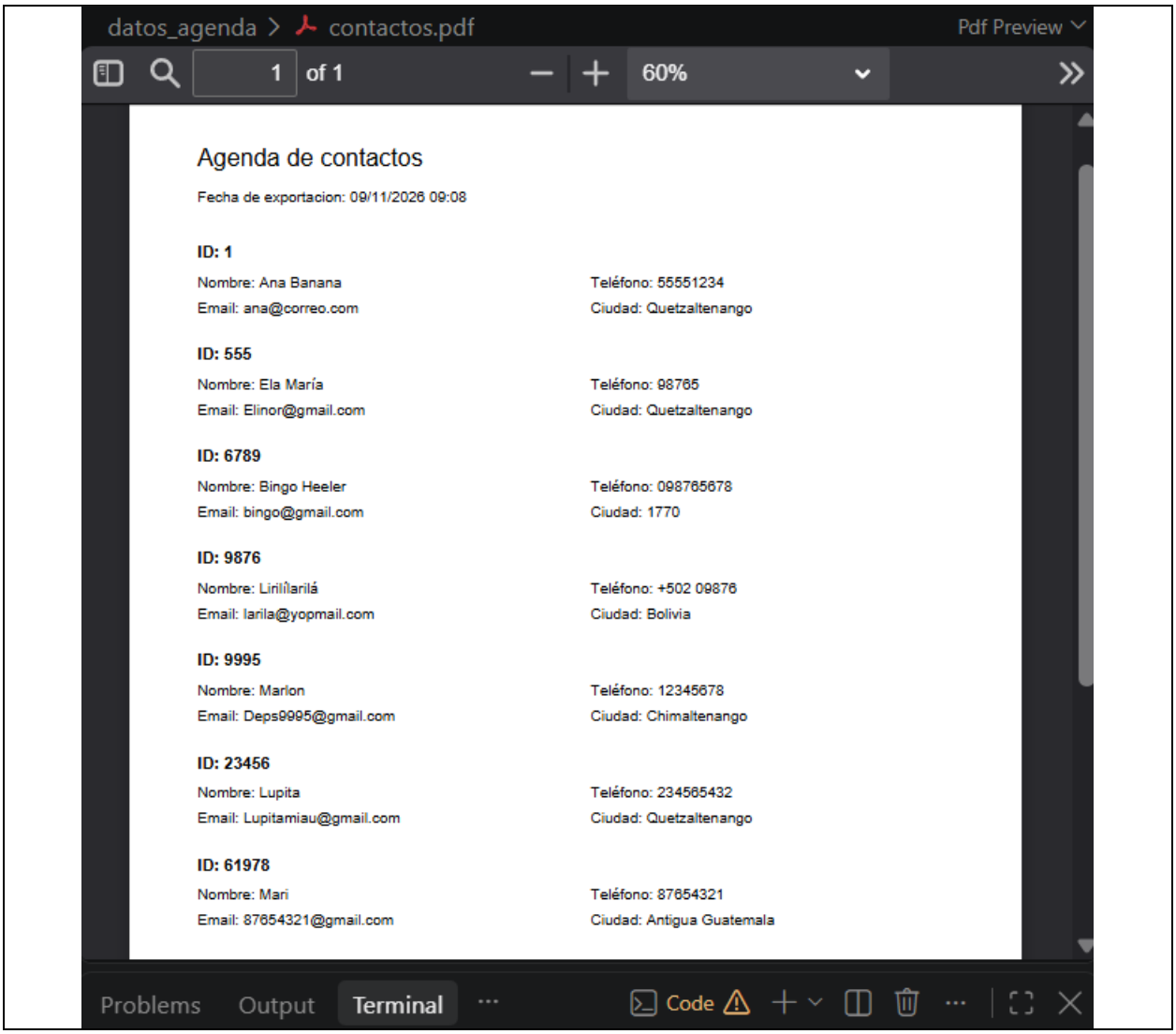

datos_agenda > {} contactos_exportados.json > { } 1
[
{
"id": 1,
"nombre": "Ana Banana",
"telefono": "55551234",
"email": "ana@correo.com",
"ciudad": "Quetzaltenango"
},
{
"id": 555,
"nombre": "Ela María",
"telefono": "98765",
"email": "Elinor@gmail.com",
"ciudad": "Quetzaltenango"
},
{
"id": 6789,
"nombre": "Bingo Heeler",
"telefono": "098765678",
"email": "bingo@gmail.com",
"ciudad": "1770"
},
{
"id": 9876,
"nombre": "Lirilílarilá",
"telefono": "+502 09876",
Problems
Output
Terminal
Code

Figura 10

Generación del reporte PDF





9. Copias de seguridad y comparación de tamaños

Para complementar el almacenamiento de información, la aplicación permite generar copias de seguridad mediante archivos ZIP. El respaldo reúne los archivos utilizados por la agenda que se encuentren disponibles al momento de su generación, incluyendo el archivo de contactos, el índice y la huella de integridad.

La finalidad de esta función es conservar una copia de los archivos importantes del sistema, de manera que exista una alternativa en caso de que los archivos originales se pierdan o

sufran algún daño. El formato ZIP permite reunir varios archivos dentro de un solo contenedor y aplicar compresión, lo que puede reducir el espacio necesario para almacenarlos.

Para evaluar el resultado de esta función, se compara el tamaño del archivo original de contactos con el tamaño del respaldo generado (Tabla 2).

Tabla 2

Tamaño del archivo de contactos y del respaldo ZIP

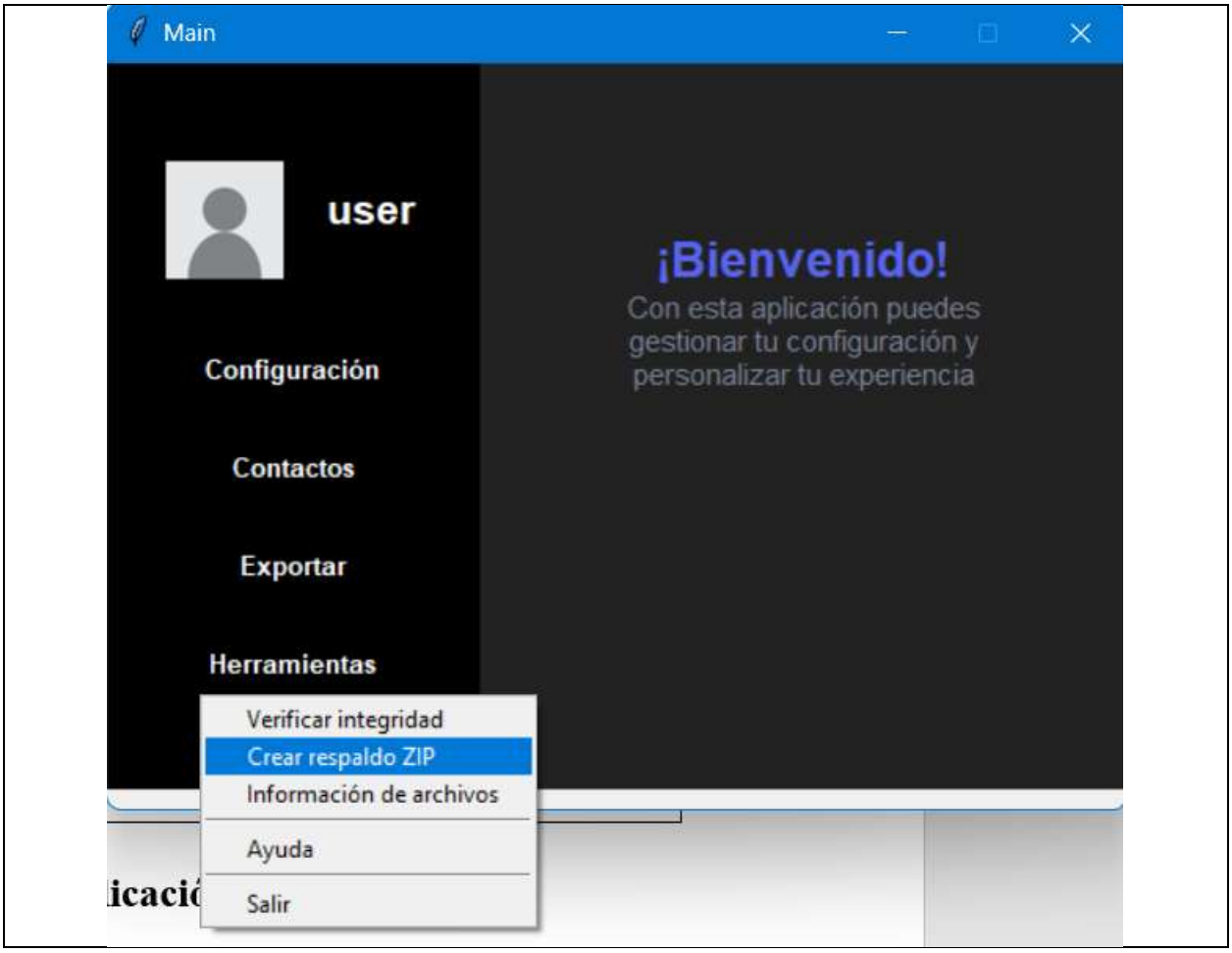
Archivo	**Tamaño**
contactos.dat	[Colocar tamaño real]
respaldo_agenda.zip	[Colocar tamaño real]

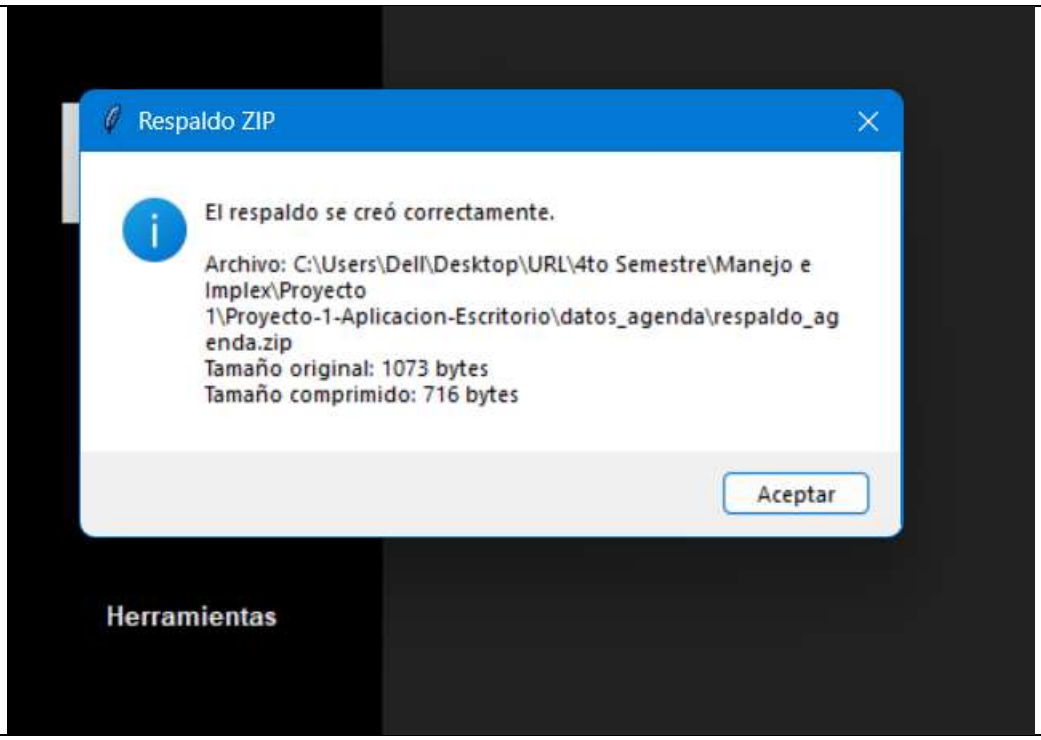
Nota. Los tamaños deben obtenerse de los archivos finales entregados.

La diferencia entre ambos tamaños permite observar el espacio que ocupa la copia de seguridad en comparación con el archivo de contactos original. Es importante considerar que el respaldo contiene más de un archivo, por lo que no debe suponerse que necesariamente ocupará menos espacio que *contactos.dat* por sí solo. El resultado depende del contenido y de la compresión aplicada.

Figura 11

Generación del respaldo ZIP o información de los archivos





10. Archivos utilizados por la aplicación

La aplicación utiliza diferentes archivos para almacenar los datos, facilitar su búsqueda y conservar copias de seguridad. Sus funciones se resumen en la Tabla 3.

Tabla 3

Archivos utilizados por la aplicación

Archivo	**Función**
contactos.dat	Almacena los registros de contactos en formato binario y con una longitud fija de 128 bytes.
indice.json	Mantiene la relación entre los identificadores de los contactos y sus números de registro.
contactos.sha256	Almacena la huella digital utilizada para comprobar la integridad del archivo de contactos.
respaldo_agenda.zip	Contiene la copia de seguridad generada por la aplicación.

Además, se generan archivos de exportación en formato JSON y PDF para permitir consultar y conservar la información en otros formatos.

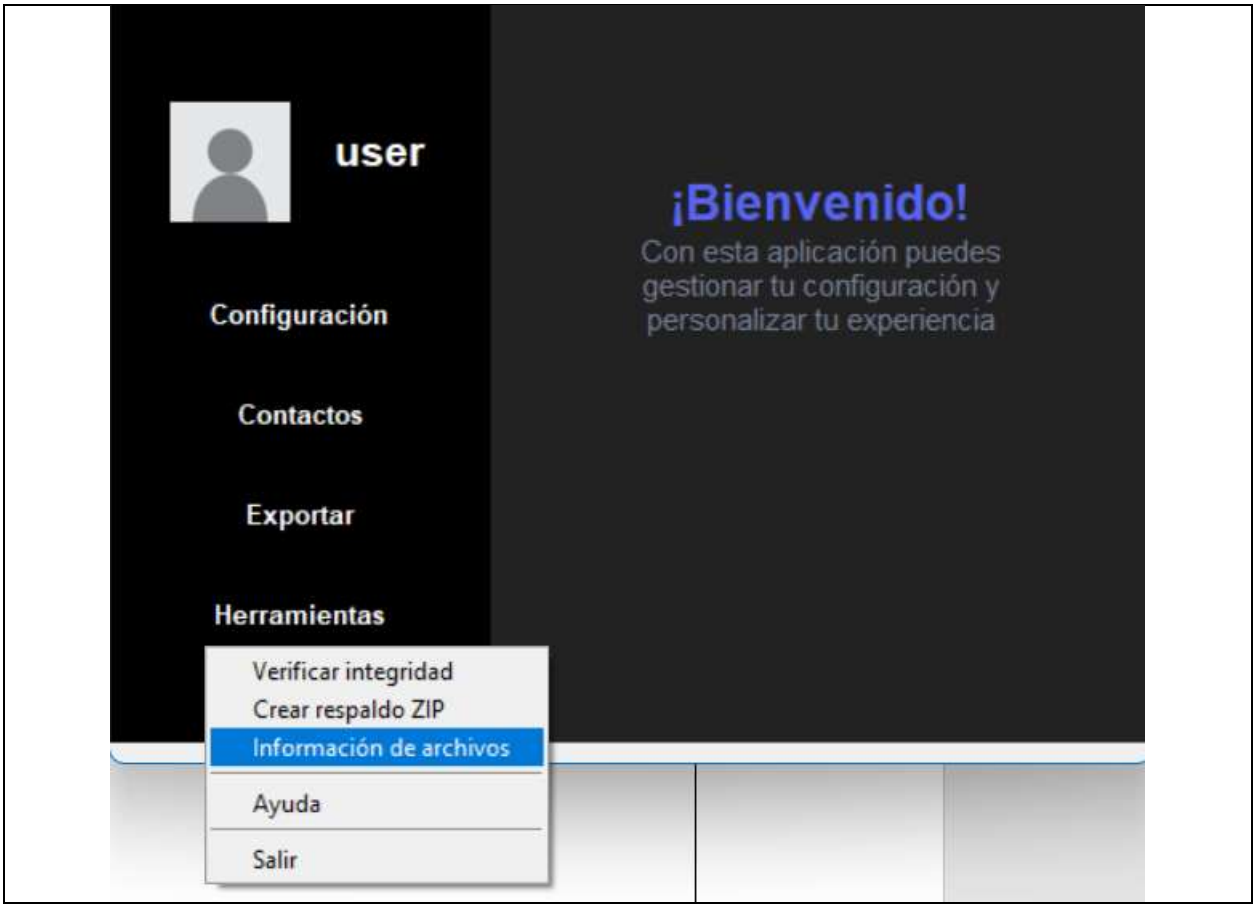
Cada archivo cumple una función específica dentro del sistema. Mientras *contactos.dat* contiene los registros principales, *indice.json* facilita su localización y *contactos.sha256* permite comprobar si el contenido del archivo de contactos ha cambiado. Por su parte, el respaldo ZIP reúne los archivos necesarios para conservar una copia de la información.

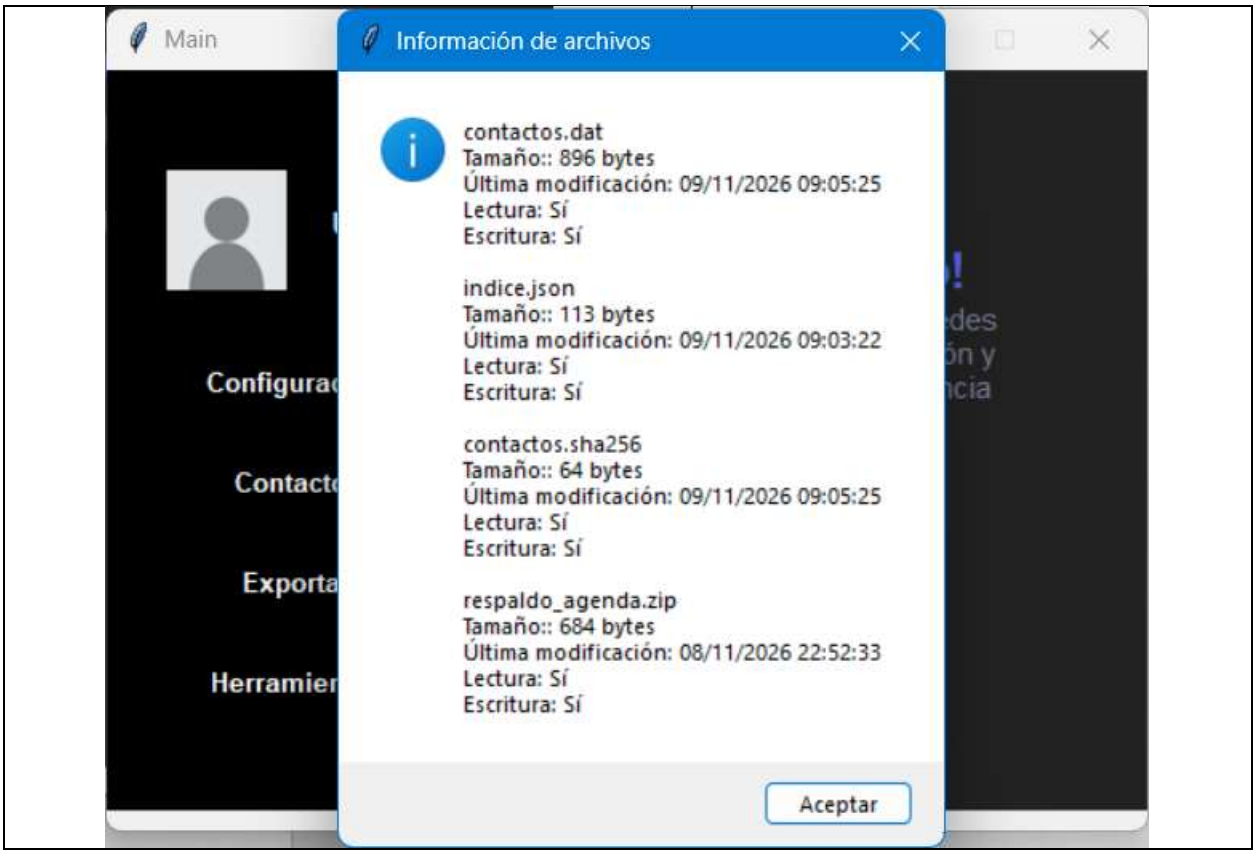
Información del archivo

La aplicación también cuenta con una ventana de información del archivo, que muestra su tamaño, la fecha de última modificación y si tiene permisos de lectura y escritura.

Figura 12

Ventana de información del archivo de contactos





11. Pruebas realizadas

Durante la revisión final se realizaron diferentes pruebas para comprobar el funcionamiento de las operaciones principales y las herramientas adicionales de la aplicación.

En primer lugar, se comprobó que la aplicación pudiera iniciarse correctamente y que su interfaz gráfica se mostrara sin inconvenientes. Posteriormente, se realizaron pruebas de registro, búsqueda y modificación de contactos. Estas operaciones permitieron verificar que la información se almacenara correctamente, pudiera recuperarse mediante el identificador y se actualizaran los datos correspondientes al modificar un registro.

También se probaron diferentes situaciones de validación. Se comprobó que la aplicación rechazara identificadores incorrectos, impidiera registrar contactos con identificadores duplicados y mostrara un mensaje comprensible cuando se intentara buscar un contacto inexistente.

En cuanto a las funciones adicionales, se comprobó que la exportación a JSON, la generación del reporte PDF y la creación de respaldos ZIP funcionaran correctamente. También se consultó la información de los archivos para comprobar que la aplicación pudiera mostrar sus datos.

Finalmente, se realizó una prueba de verificación de integridad, mediante la cual se comprobó que la aplicación pudiera detectar una modificación manual del archivo de contactos. Después de las pruebas, se verificó que los archivos conservaran su integridad. Los resultados obtenidos permitieron confirmar el funcionamiento de las principales características implementadas en la agenda.

Capturas de pantalla del funcionamiento de la aplicación

Figura 13

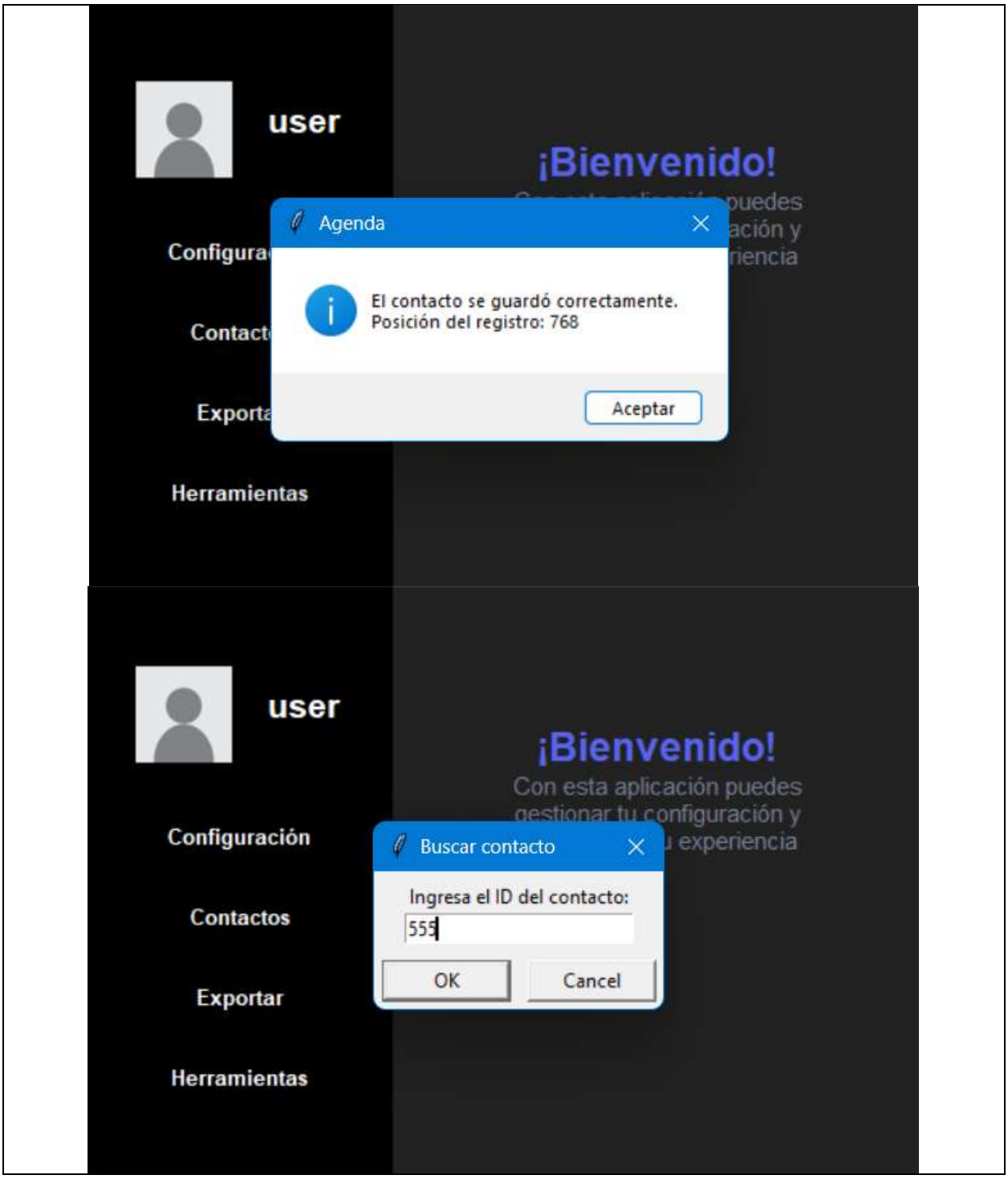
Ventana principal y menú de la aplicación

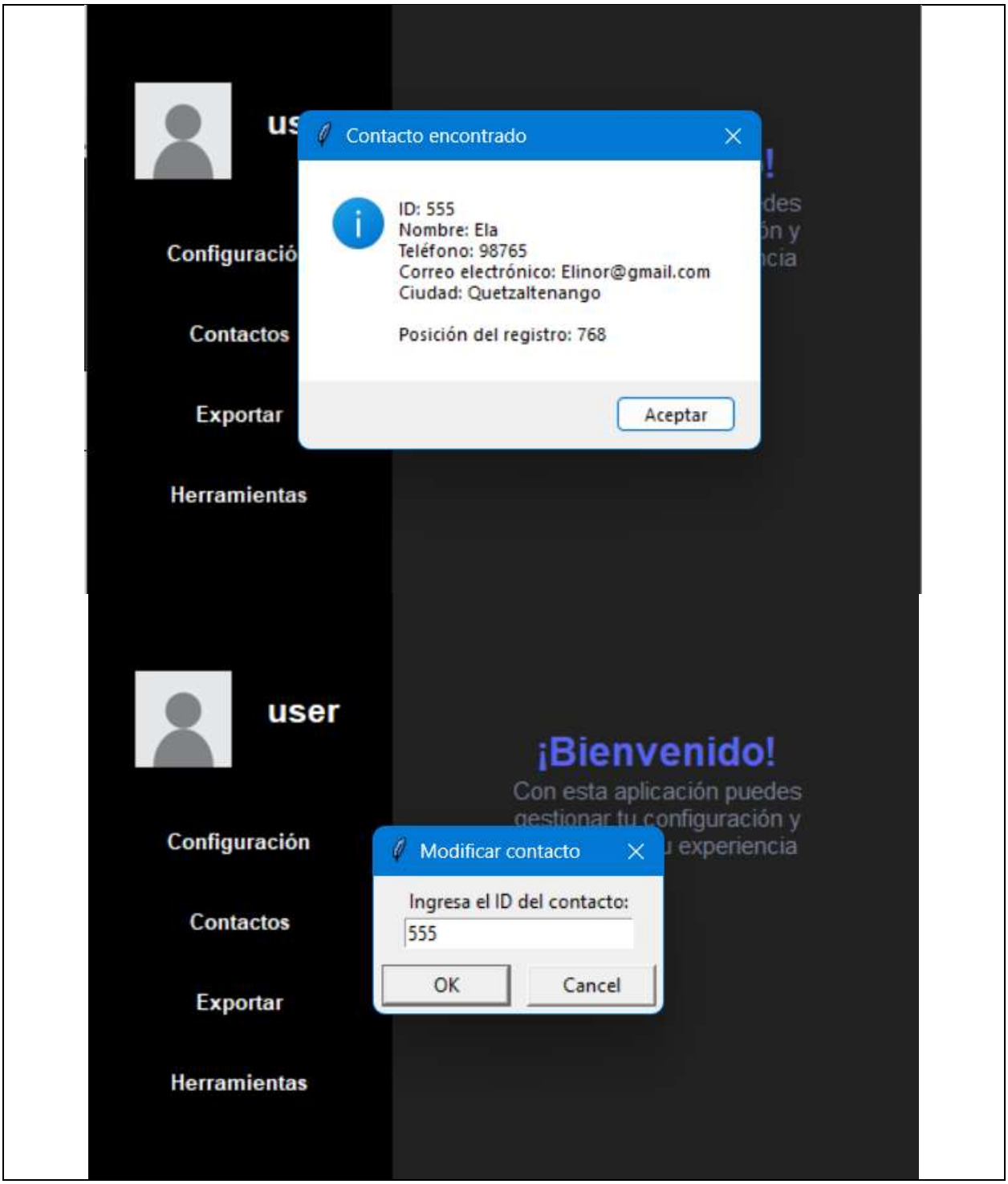


Figura 14

Agregar, buscar por identificador y modificar contactos




user
¡Bienvenido!
Agenda
El contacto se guardó correctamente.
Posición del registro: 768
Aceptar
Herramientas
user
¡Bienvenido!
Con esta aplicación puedes
Configuración
Buscar contacto
Ingresa el ID del contacto:
555
OK
Cancel
Contactos
Exportar
Herramientas


Contacto encontrado
ID: 555
Nombre: Ela
Teléfono: 98765
Correo electrónico: Elinor@gmail.com
Ciudad: Quetzaltenango
Posición del registro: 768
Aceptar
Configuración
Contactos
Exportar
Herramientas
user
¡Bienvenido!
Con esta aplicación puedes
Modificar contacto
Ingresa el ID del contacto:
555
OK
Cancel

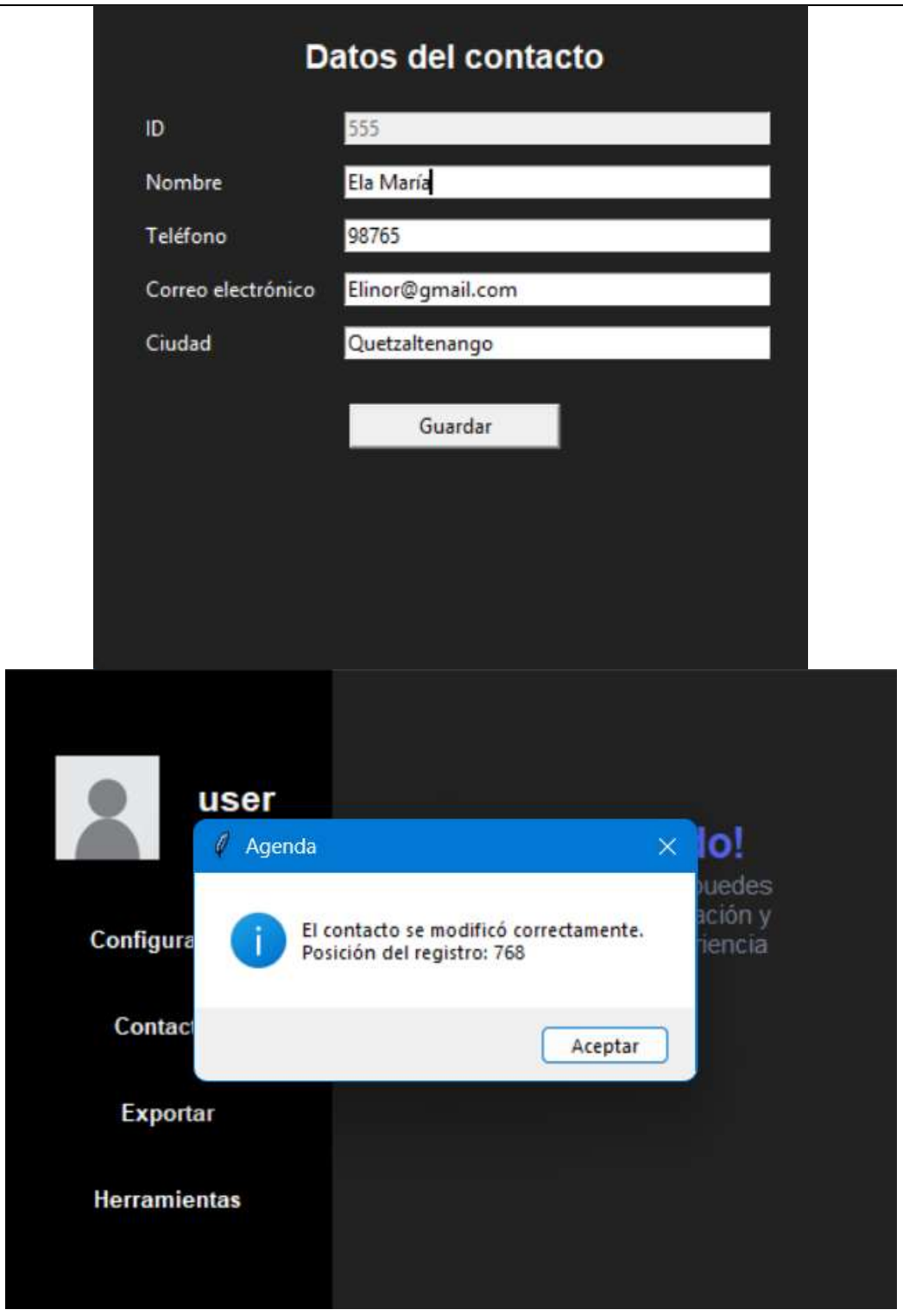
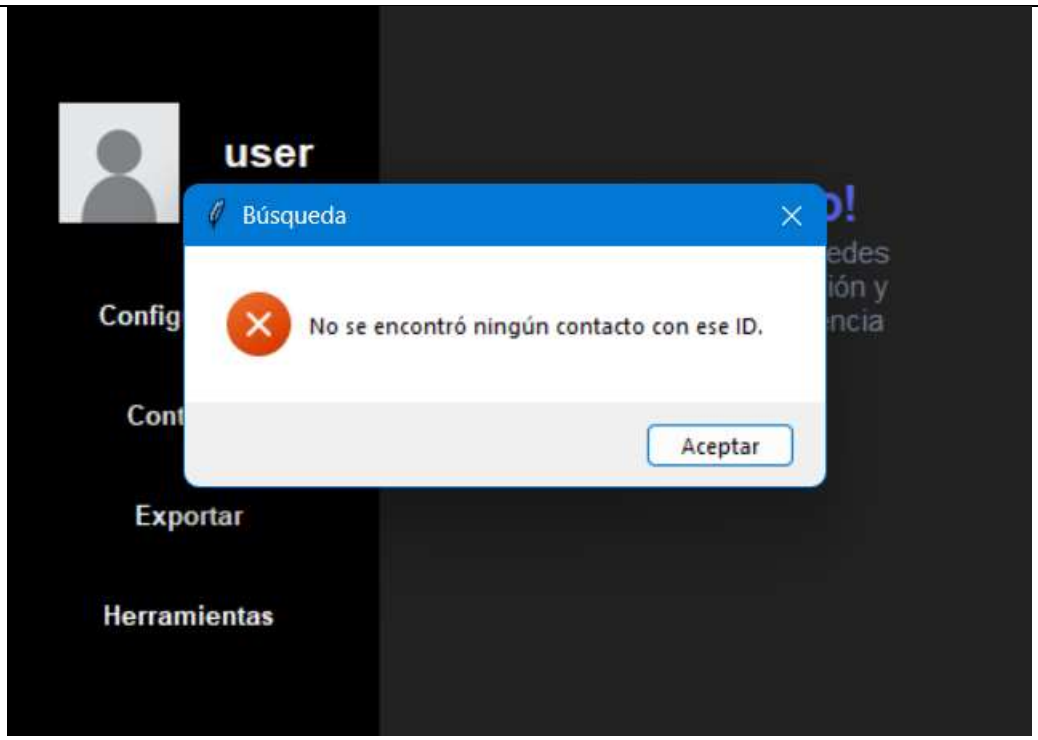

Datos del contacto
ID
555
Nombre
Ela María
Teléfono
98765
Correo electrónico
Elinor@gmail.com
Ciudad
Quetzaltenango
Guardar
user
Agenda
El contacto se modificó correctamente.
Posición del registro: 768
Aceptar
Configura
Contac
Exportar
Herramientas

Figura 15

Mensajes de validación de la aplicación

- **Contacto inexistente:** busca un ID que no esté registrado. Muestra el mensaje “No se encontró ningún contacto con ese ID.”





- **ID duplicado:** Agregar un contacto con un ID que ya existe. El mensaje es “Ya existe un contacto registrado con ese ID.”




Datos del contacto
ID
9995
Nombre
Nombre
Teléfono
65432553
Correo electrónico
correo@gmail.com
Ciudad
Ciudad
Guardar


Datos del contacto
ID
9995
No se pudo guardar el contacto.
Ya existe un contacto registrado con ese ID.
Aceptar

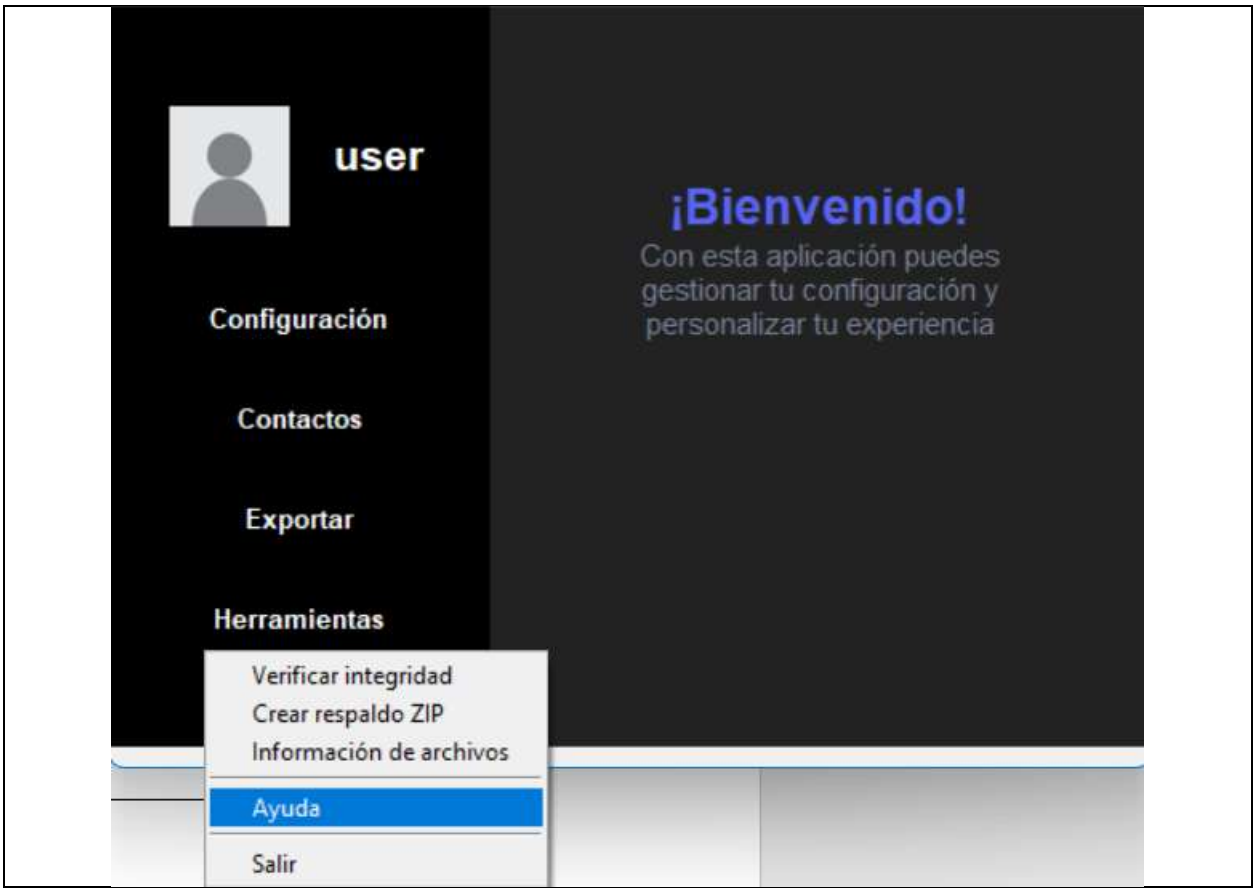
- **ID incorrecto:** Agregar un contacto con un ID inválido, como letras o un campo vacío.

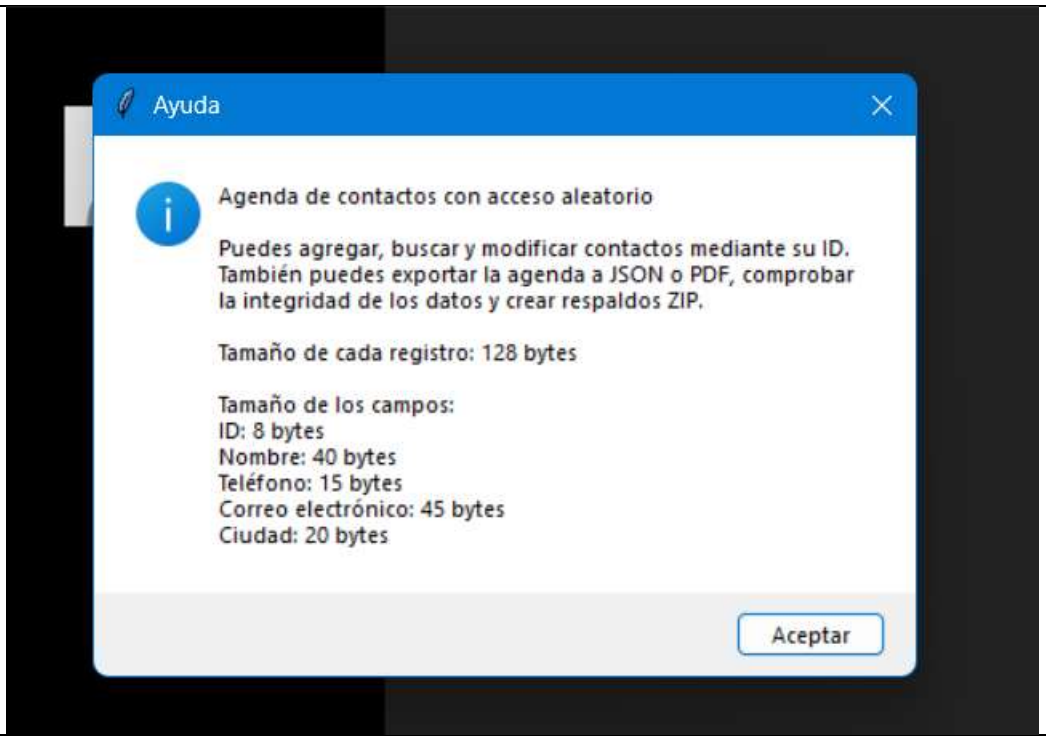




Figura 16

Apartado de ayuda de la aplicación




Ayuda
Agenda de contactos con acceso aleatorio
Puedes agregar, buscar y modificar contactos mediante su ID.
También puedes exportar la agenda a JSON o PDF, comprobar
la integridad de los datos y crear respaldos ZIP.
Tamaño de cada registro: 128 bytes
Tamaño de los campos:
ID: 8 bytes
Nombre: 40 bytes
Teléfono: 15 bytes
Correo electrónico: 45 bytes
Ciudad: 20 bytes
Aceptar